

智学灵境 EduVerse

高等教育个性化学习多智能体系统自治说明文档

版本: v2.0 文档日期: 2026-07-19

摘要

智学灵境 **EduVerse** 是面向高校人工智能、计算机与电子信息类课程的个性化学习多智能体系统。当前高校工科专业存在课程知识体系繁杂、学生基础分层明显、统一化教学资源难以适配差异化认知节奏、实操训练与理论讲解脱节、学习过程数据无法闭环复用等痛点，平台以学生全维度画像为核心入口，以多智能体协同资源生成、自适应学习路径规划、分层题库智能训练、7×24 小时场景化智能辅导和全过程学习效果量化评估为核心能力，打破传统在线教学平台资源固化、千人一面、过程无感知的局限，把传统课程静态、统一、不可追踪的资料集合，改造成可实时捕捉学习行为、动态迭代内容难度、适配个人认知水平的专属交互式学习空间。

系统前端采用 **React**、**TypeScript**、**Vite** 与 **Tailwind CSS** 构建轻量化高性能交互界面，搭配组件化封装实现课程文档、编程实操、可视化图谱、习题作答多场景统一渲染；状态层使用 **Zustand** 轻量化全局状态管理方案，统一存储、更新与持久化学生画像标签、时序化学习行为记录、AI 资源异步生成状态、分模块题库训练数据与错题溯源信息，保障前端流式交互无卡顿、数据同步低延迟。智能体层创新搭建“灵境队长、解构者、编织者、出题官、视觉师、实操匠、校验官”七类专业智能体角色分工协同完整学习服务流程，其中灵境队长统筹全局学习任务，解构者拆解知识难点并关联知识图谱节点，编织者整合图文、代码、课件生成定制化学习素材，出题官基于易错点动态组卷，视觉师生成算法可视化、电路仿真图解，实操匠提供代码调试、硬件实操分步指导，校验官负责内容真实性、知识点准确性、AI 生成幻觉拦截；各智能体通过中央编排器完成复杂学习任务自动拆解、多阶段任务调度、全流程操作日志记录、大模型内容流式输出与多层级内容安全校验，从源头降低 AI 生成内容失真、知识点错误问题。

与传统静态资源型、单轮推荐式学习平台相比，智学灵境 **EduVerse** 的核心差异化关键价值在于构建统一连续的学生学习状态空间，将“学习目标、认知偏好、知识薄弱易错点、学习节奏、科研兴趣方向、阶段性学习结果、实操完成度、答疑交互记录”八大维度数据持续纳入统一建模体系。学生每一次课程阅读、智能答疑交互、习题作答实操、多媒体资源调取行为都会实时回流至学生画像模型，反向迭代优化知识推送逻辑与学习路径难度梯度，形成“行为采集 — 画像更新 — 资源适配 — 学习反馈”的闭环自适应机制，让系统摆脱传统平台一次性静态内容推荐的局限，转变为能够长期追踪学习成长、动态适配能力变化、持续迭

代辅导策略的伴随式智能学习伙伴。平台可广泛应用于人工智能、软件工程、电子信息、自动化等本科、研究生工科课程日常教学、课后自主复习、课程实验实训、考前专项强化等场景，为高校教师提供学情批量分析、班级薄弱知识点定位、个性化教学素材批量生成工具，同时为学生提供无间断、高度适配个人能力的自主学习支撑，助力高校工科专业实现规模化个性化智能教学落地。

关键词：个性化学习，多智能体协同，学习数字画像，大模型流式生成，学科知识图谱，AI 内容防幻觉校验，前端工程化，**React**，**TypeScript**，自适应学习闭环，工科智能教学

目 录

一、项目概述	1
1.1 项目背景与痛点	1
1.2 项目目标	1
1.3 功能概览	2
二、系统总体设计	4
2.1 整体架构	4
2.2 技术栈选型	4
2.3 数据模型	5
三、核心功能与代码实现	7
3.1 工作台模块	7
3.2 学习画像模块	9
3.3 学习计划模块	11
3.4 资源工坊模块	12
3.5 学习路径模块	16
3.6 题库训练模块	18
3.7 智能辅导模块	19
3.8 学习评估模块	21
3.9 资源中心模块	23
四、模块详细设计	26
五、关键算法说明	29
5.1 画像增量更新算法	29
5.2 知识点匹配算法	30

5.3 资源生成编队算法	31
5.4 评估统计算法	31
六、智能体协同机制	33
6.1 角色定义	33
6.2 编排器状态流转	33
6.3 流式输出机制	34
七、安全校验与防幻觉	35
八、系统测试	36
8.1 测试环境	36
8.2 功能测试结果	36
8.3 性能与稳定性	37
九、AI 智能引入说明	38
9.1 本项目使用的开源依赖均遵循其原始协议，主要依赖包括：	38
9.2 AI Coding 工具	38
十、应用前景与迭代规划	39
10.1 应用场景	39
10.2 短期迭代	39
10.3 中期迭代	39
10.4 长期规划	39

一、项目概述

1.1 项目背景与痛点

高校课程正在快速引入 AI、大模型、编程实践与跨学科项目，但教学组织仍以统一课程表、统一资料包和统一测验为主。对于基础薄弱、偏好图解、偏好代码实操、准备考研或准备项目竞赛的学生来说，同一套学习材料无法同时满足不同目标和节奏。

原 loAGuard 文档强调“自然语言意图到可执行任务”的闭环，本项目采用相同的自治设计思想：学生不需要理解复杂的资源组织逻辑，只需要表达“我想学什么、卡在哪里、想以什么方式学”，系统就能组织智能体完成画像更新、资源生成、路径调整和结果评估。

表 1.1 痛点分析

痛点	表现	智学灵境的对应解决方式
资源繁杂	课程资料、视频、题库、代码散落在不同平台	资源中心统一收纳，按知识点、类型、难度和画像标签检索
学习路径固定	标准课表难以匹配不同基础与目标	学习路径根据画像、练习结果和节点状态动态调优
反馈滞后	学生做完题后很难知道下一步补什么	评估模块生成雷达图、热力图和资源推送建议
内容可信度不足	AI 生成内容可能出现幻觉或不适合课程范围	校验官执行敏感词、知识点白名单、代码块和引用检查

1.2 项目目标

项目目标是构建一个“对话建档、多模态生成、路径自适应、结果可评估”的高等教育学习平台。系统既要能展示完整的产品体验，也要能通过源码体现核心技术路径，便于竞赛评审、课程验收和后续扩展。

- 对话建档：通过首次引导和随学随新的自然语言输入，构建六维学习画像。
- 多模态生成：通过七智能体协作生成讲解文档、思维导图、题库、拓展阅读、教学动画和代码实操。

- 路径自适应：以知识图谱节点为基本单位，根据画像和学习记录刷新学习路径。
- 结果可评估：将练习正确率、学习记录、资源使用和画像变化汇总为可解释的学习反馈。

1.3 功能概览

表 1.2 功能概览

模块	入口页面	核心职责
工作台	Home.tsx	汇总学习仪表盘、智能体编队、快捷入口和今日状态
学习画像	Profile.tsx	构建六维画像、展示雷达图、维护画像版本
学习计划	StudyPlan.tsx	生成阶段性计划、安排每日任务、跟踪完成情况
资源工坊	Workshop.tsx	触发多智能体生成资源并展示协作日志
学习路径	LearningPath.tsx	展示知识路径图谱、节点状态和资源抽屉
题库训练	Practice.tsx	提供单选、多选、填空、判断、代码补全等训练
智能辅导	Tutor.tsx	基于知识库匹配问题并流式输出结构化讲解
学习评估	Assessment.tsx	用雷达图、热力图和建议展示学习效果
资源中心	Library.tsx	统一管理已生成资源、收藏与筛选

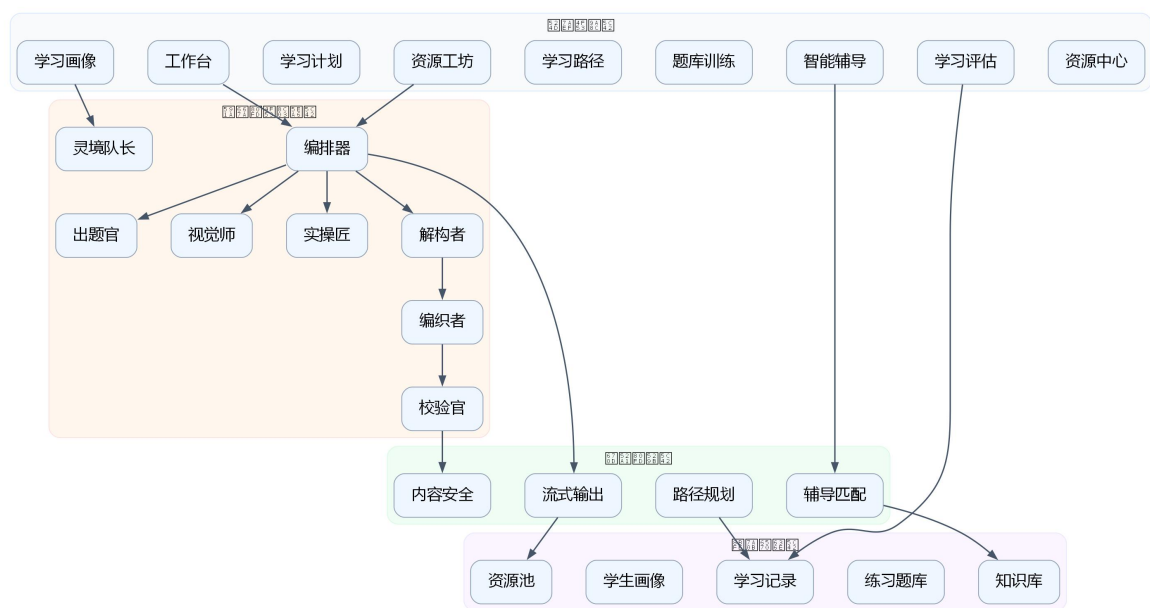


图 1.1 智学灵境系统分层架构图

二、系统总体设计

2.1 整体架构

系统采用前端体验层、多智能体调度层、服务能力层、课程数据层四层结构。前端负责页面呈现和交互状态，调度层负责把学习任务转化为智能体流水线，服务层提供流式输出、安全过滤与辅导匹配，数据层保存课程知识库、画像、资源池、练习题和学习记录。

当前演示版本采用前端本地数据与剧本式流式回放，保证评审时一键启动、稳定演示。生产环境可在 **AgentService** 抽象下接入真实大模型、后端 **API** 和数据库，页面层与智能体编排逻辑不需要大规模重写。

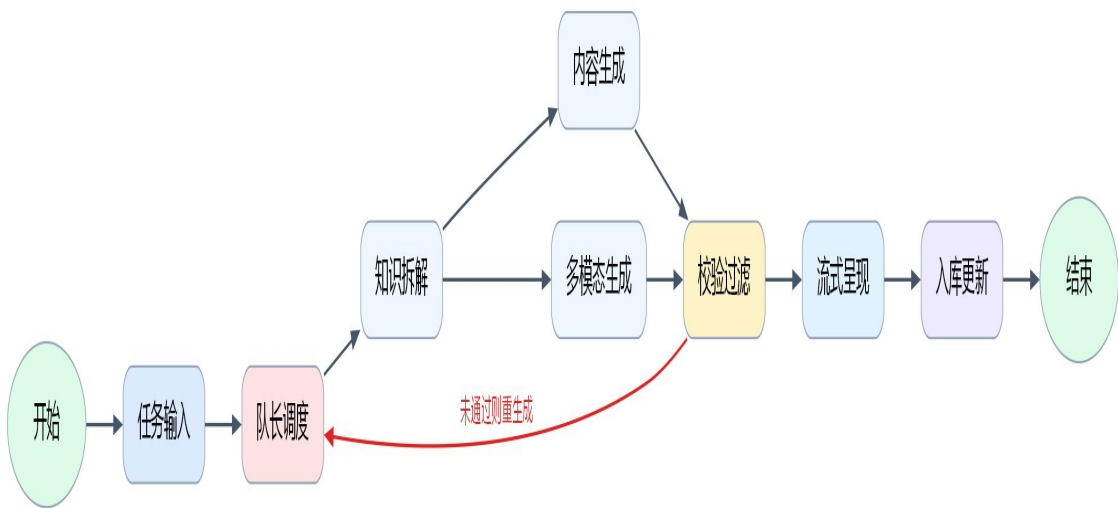


图 2.1 多智能体自治工作流

2.2 技术栈选型

表 2.1 技术栈选择

技术分类	组件	选型理由
前端框架	React 18 + TypeScript + Vite	组件化开发效率高，类型约束明确，适合竞赛演示快速迭代
样式与动效	Tailwind CSS + framer-motion	便于实现玻璃态卡片、粒子背景、流式动画和主题切换

状态管理	Zustand	轻量、Hook 友好，适合管理画像、资源、练习和对话状态
可视化	ReactFlow / SVG / Plotly 思路	支持路径图谱、雷达图、热力图和思维导图展示
内容渲染	react-markdown + remark-gfm	支持 Markdown、表格、代码块和数学内容呈现
智能体层	Orchestrator + AgentDefinition	用角色化智能体组织任务阶段，便于展示协作过程
安全过滤	SafetyService	实现敏感词、白名单、代码块与引用检查，体现防幻觉机制

2.3 数据模型

数据模型围绕学习者、画像版本、知识节点、学习资源、学习记录和学习计划展开。系统没有把画像作为静态表单，而是作为可版本化对象存储，每次对话、练习或计划变更都可以生成新的画像证据。

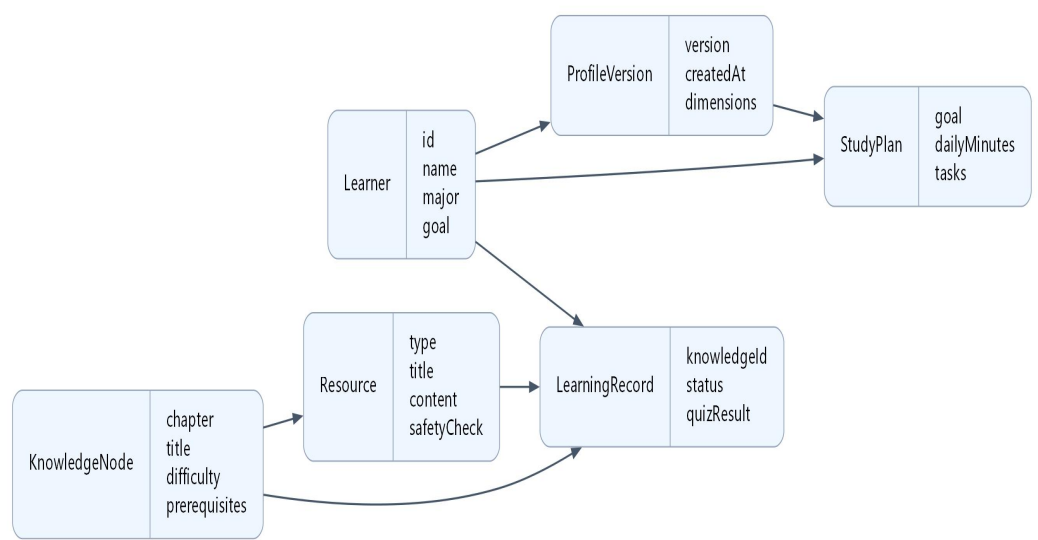


图 2.2 核心数据关系图

核心代码如下：

```
export interface ProfileVersion {
```

```
    id: string;
    version: number;
    createdAt: string;
    trigger: string;
    dimensions: ProfileDimension[];
}

export interface Resource {
    id: string;
    type: ResourceType;
    title: string;
    knowledgeId: string;
    content: string;
    metadata: ResourceMetadata;
    safetyCheck: { passed: boolean; flags: string[]; score: number };
}
```

三、核心功能与代码实现

3.1 工作台模块

对应文件：Home.tsx / AppShell.tsx / Sidebar.tsx

工作台是学生进入系统后的总览页面，承担学习状态汇总、智能体编队展示、快捷任务发起和今日学习提醒四类职责。截图中的“今日学习、连续打卡、平均掌握、待学节点”等卡片，就是工作台把多源状态转化为可读指标的呈现方式。

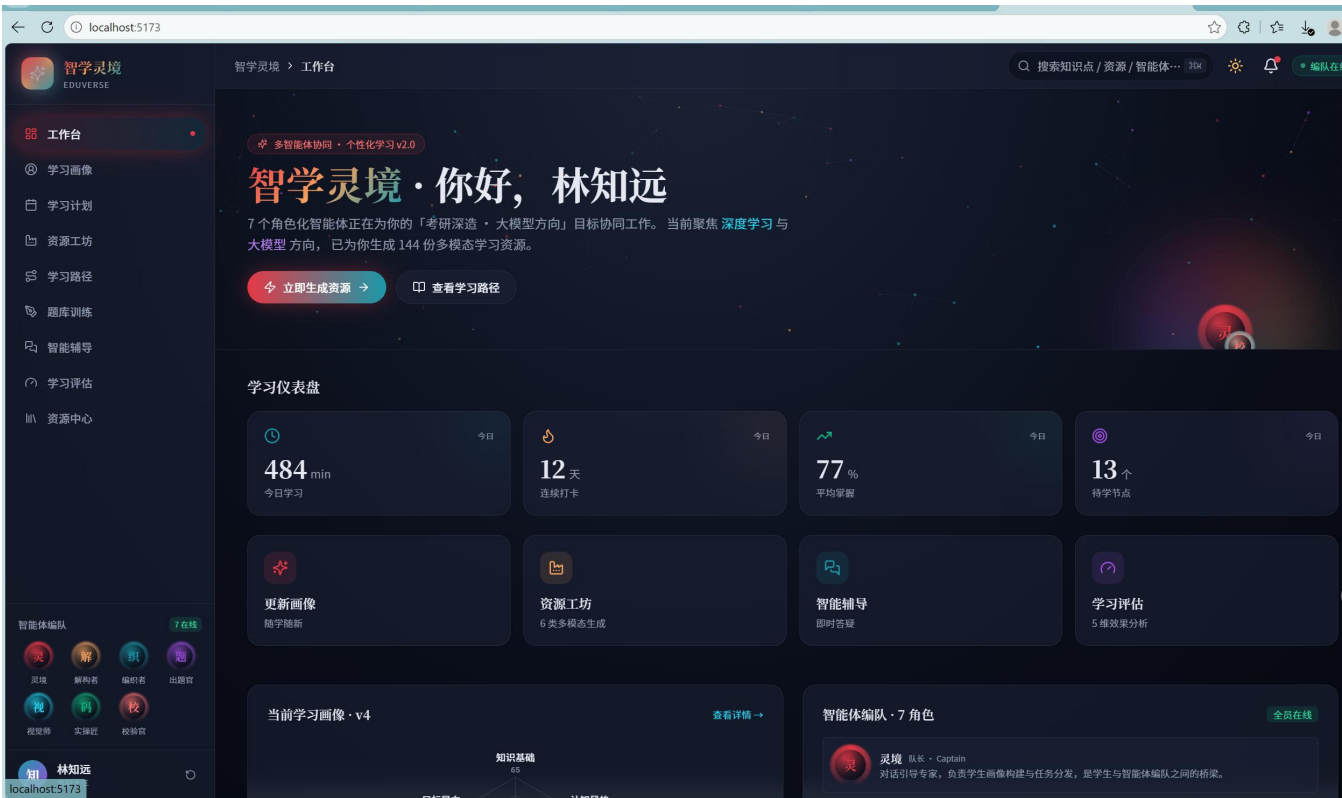


图 3.1 工作台首页界面截图

智学灵境 EduVerse 整体架构分为前端业务层、多智能体协同层、服务调度层与底层数据支撑层四层，完整实现工科课程个性化学习全流程闭环。

顶层前端业务层面向学生提供九大功能模块入口，包含学习画像、工作台、学习计划、资源工坊、学习路径、题库训练、智能辅导、学习评估、资源中心，覆盖课前规划、课中学习、课后练习、学情复盘全场景；所有前端交互请求统一汇聚至多智能体协同层的核心调度单元——编排器，由编排器承接上层业务指令，统筹调度七类分工明确的智能体角色。

多智能体集群包含灵境队长、出题官、视觉师、实操匠、解构者、编织者、校验官：灵境队长承接学习画像的基础学情数据，统筹整体学习任务；出题官、视觉师、实操匠分别负

责习题生成、算法 / 电路可视化素材制作、编程与硬件实操指导；解构者拆解知识图谱难点节点，交由编织者整合课件、图文、代码等定制化学习素材，最终由校验官完成知识点正误、AI 幻觉拦截等内容安全核验。

编排器同步驱动四大核心调度服务：内容安全、流式输出、路径规划、辅导匹配。其中经校验官过滤后的合规学习素材通过流式输出推送至资源池；路径规划服务结合学生画像动态生成适配认知水平的专属学习路线；辅导匹配模块对接智能辅导前端入口，依据学习记录、练习题库、知识库实时推送针对性答疑内容。

底层数据层由资源池、学生画像、学习记录、练习题库、知识库构成统一数据底座，全链路学习行为、AI 生成资源、答题错题、知识掌握程度持续回流更新学生画像，反向迭代上层智能体任务调度与资源生成逻辑，形成“前端交互 - 多智能体协同调度 - 服务处理 - 数据反馈画像”的自适应闭环，为计算机、人工智能、电子信息类高校课程提供伴随式个性化智能学习支撑。

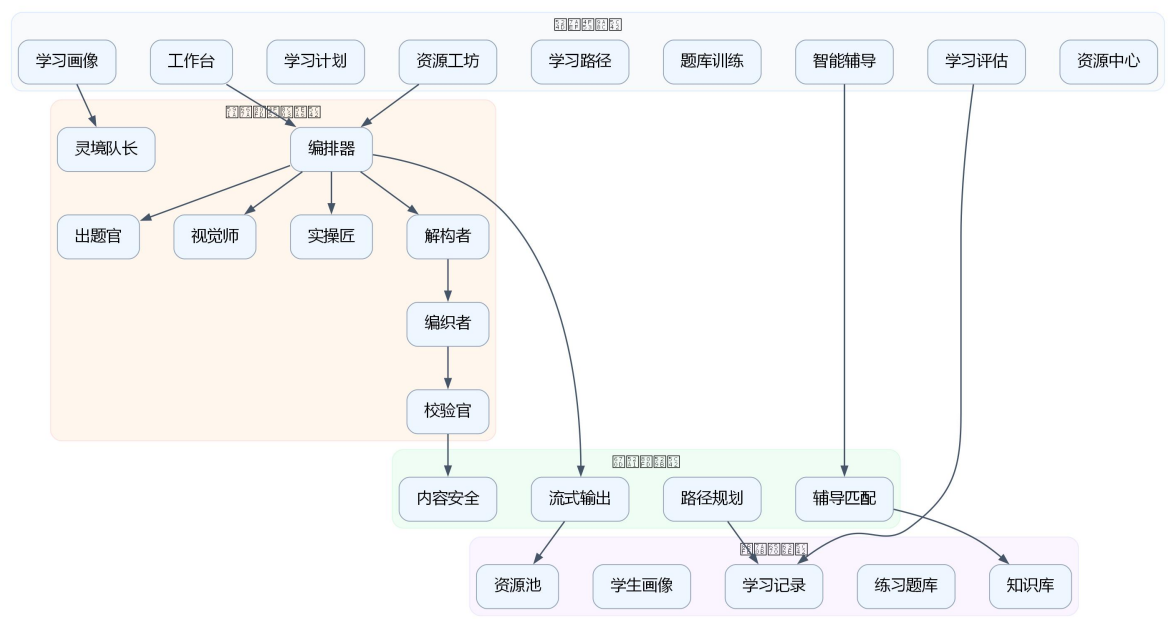


图 3.2 工作台模块处理流程图

核心代码说明：该片段摘取或概括自当前项目源码，体现模块实现的关键状态、流程或算法。

```
const navItems = [
  { path: "/", label: "工作台" },
  { path: "/profile", label: "学习画像" },
```

```

{ path: "/plan", label: "学习计划" },
{ path: "/workshop", label: "资源工坊" },
{ path: "/path", label: "学习路径" },
{ path: "/practice", label: "题库训练" },
{ path: "/tutor", label: "智能辅导" },
{ path: "/assessment", label: "学习评估" },
{ path: "/library", label: "资源中心" },
];

```

功能运行时，页面层只负责采集用户操作和展示结果，核心状态变更集中由 **Store** 或服务函数处理。这样的划分让页面逻辑保持轻量，也便于后续接入真实后端接口、大模型服务或学习数据仓库。

3.2 学习画像模块

对应文件：Profile.tsx / useAppStore.ts / learner.ts

学习画像模块通过对话和学习行为维护知识基础、认知风格、易错偏好、学习节奏、兴趣方向、目标导向六个维度。画像不是一次填写后固定不变，而是借助 **regenerateProfile** 与 **predictProfileChanges** 在学生表达“学会、卡住、太难、想学大模型”等信号时自动更新。

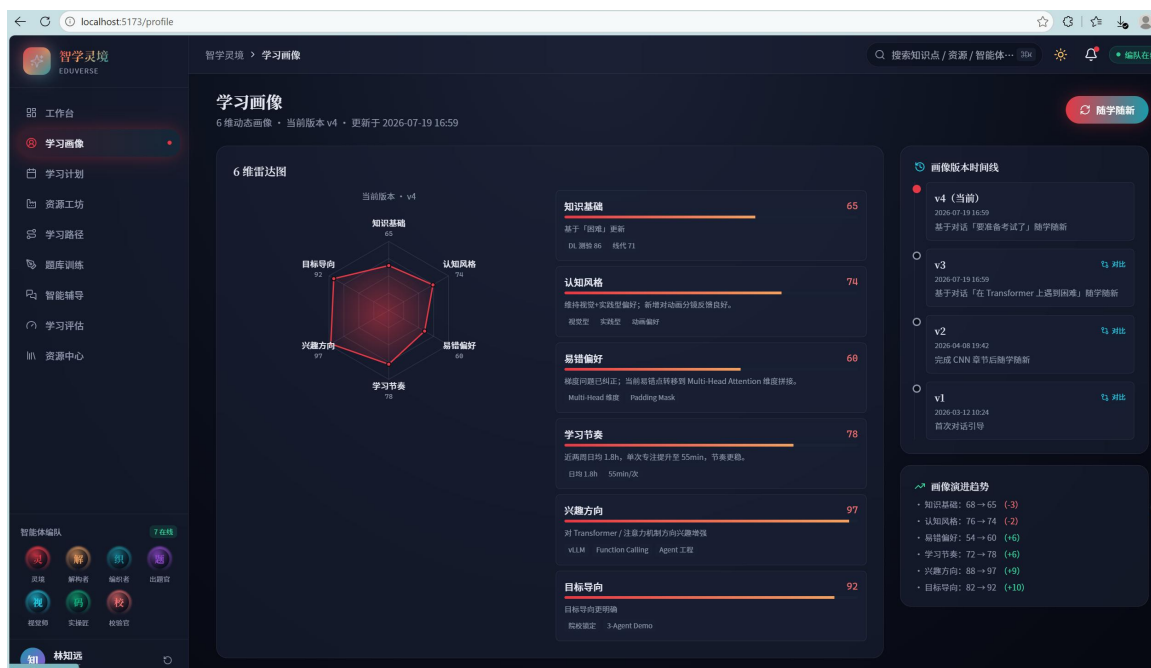


图 3.3 学习画像界面截图

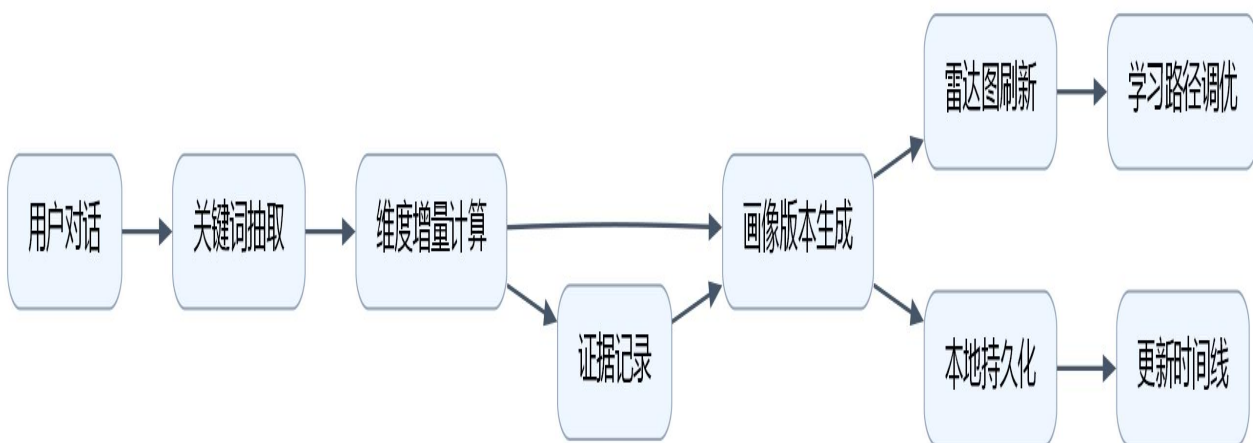


图 3.4 学习画像模块处理流程图

核心代码说明：该片段摘取或概括自当前项目源码，体现模块实现的关键状态、流程或算法。

```

function analyzeText(text: string) {
  const dimensionKeywords = {
    知识基础: { up: ["掌握", "学会"], down: ["不会", "卡住"] },
    学习节奏: { up: ["加快", "冲刺"], down: ["放慢", "压力大"] },
    兴趣方向: { up: ["大模型", "Transformer", "Agent"], down: ["不感兴趣"] },
  };
  const dimensionDeltas = {};
  for (const [dimName, config] of Object.entries(dimensionKeywords)) {
    let delta = 0;
    for (const kw of config.up) if (text.includes(kw)) delta += 5;
    for (const kw of config.down) if (text.includes(kw)) delta -= 5;
    dimensionDeltas[dimName] = delta;
  }
  return dimensionDeltas;
}

```

功能运行时，页面层只负责采集用户操作和展示结果，核心状态变更集中由 **Store** 或服务函数处理。这样的划分让页面逻辑保持轻量，也便于后续接入真实后端接口、大模型服务或学习数据仓库。

3.3 学习计划模块

对应文件：`StudyPlan.tsx` / `practice.ts` / `useAppStore.ts`

学习计划模块把学习目标、重点方向、难度、每日学习时长和持续天数转化为阶段性任务。它不是简单日历，而是把任务、里程碑和知识点关联起来，服务于考研复习、课程补强、项目实操等不同目标。

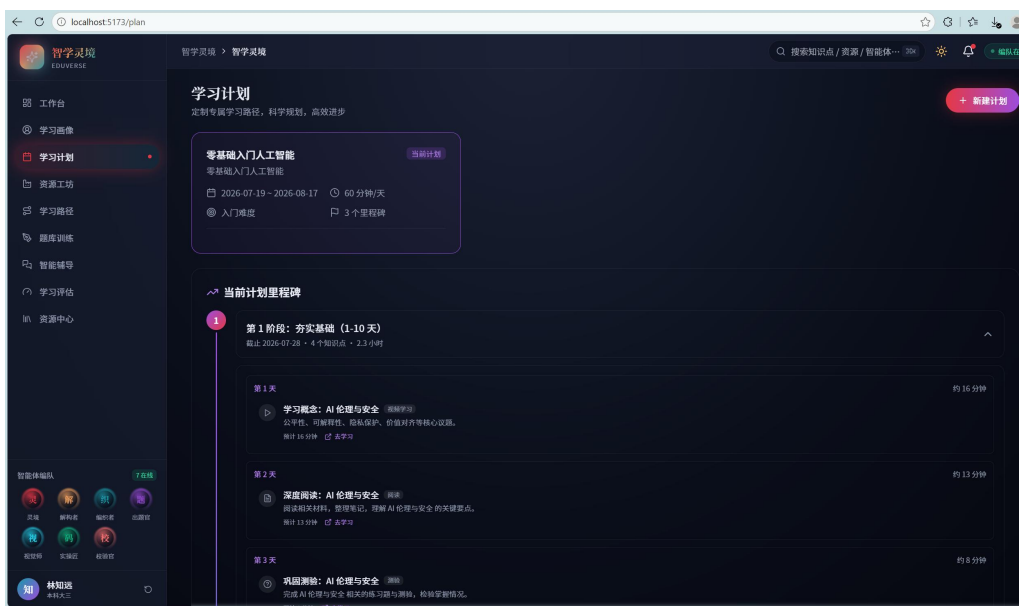


图 3.5 学习计划界面截图



图 3.6 新增学习计划界面截图

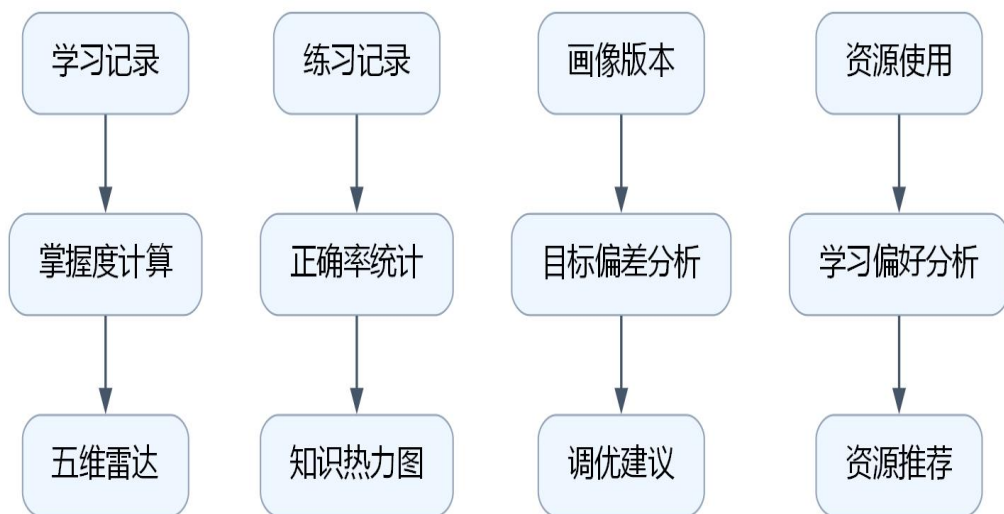


图 3.7 学习计划模块处理流程图

核心代码说明：该片段摘取或概括自当前项目源码，体现模块实现的关键状态、流程或算法。

```

createStudyPlan: (options) => {
  const plan = generateStudyPlan(options, get().currentProfile,
get().learningRecords);
  set((state) => ({
    studyPlans: [plan, ...state.studyPlans],
    currentPlanId: plan.id,
  }));
  return plan;
}

```

功能运行时，页面层只负责采集用户操作和展示结果，核心状态变更集中由 **Store** 或服务函数处理。这样的划分让页面逻辑保持轻量，也便于后续接入大模型服务或学习数据仓库。

3.4 资源工坊模块

对应文件：Workshop.tsx / orchestrator.ts / agents.ts / resources.ts

资源工坊是多智能体协作最集中的模块。学生选择资源类型和主题后，编排器根据资源类型匹配智能体编队，逐阶段输出日志，并在校验通过后把内容流式呈现给学生。它覆盖讲解文档、思维导图、题库、拓展阅读、教学动画和代码实操六类资源。

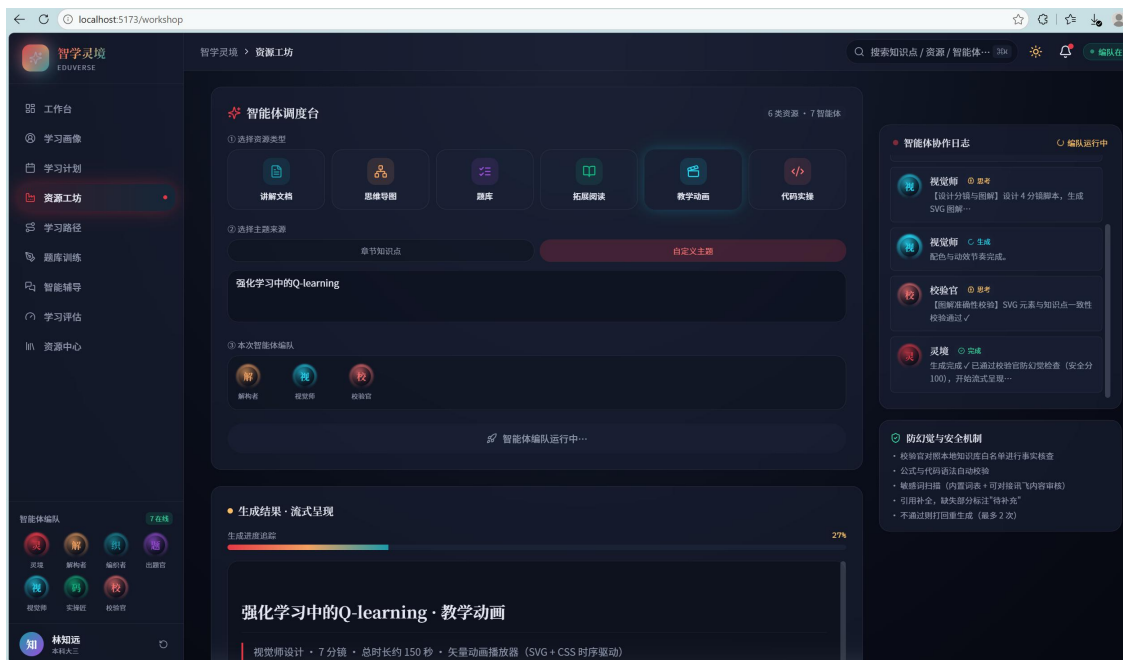


图 3.8 资源工坊界面截图

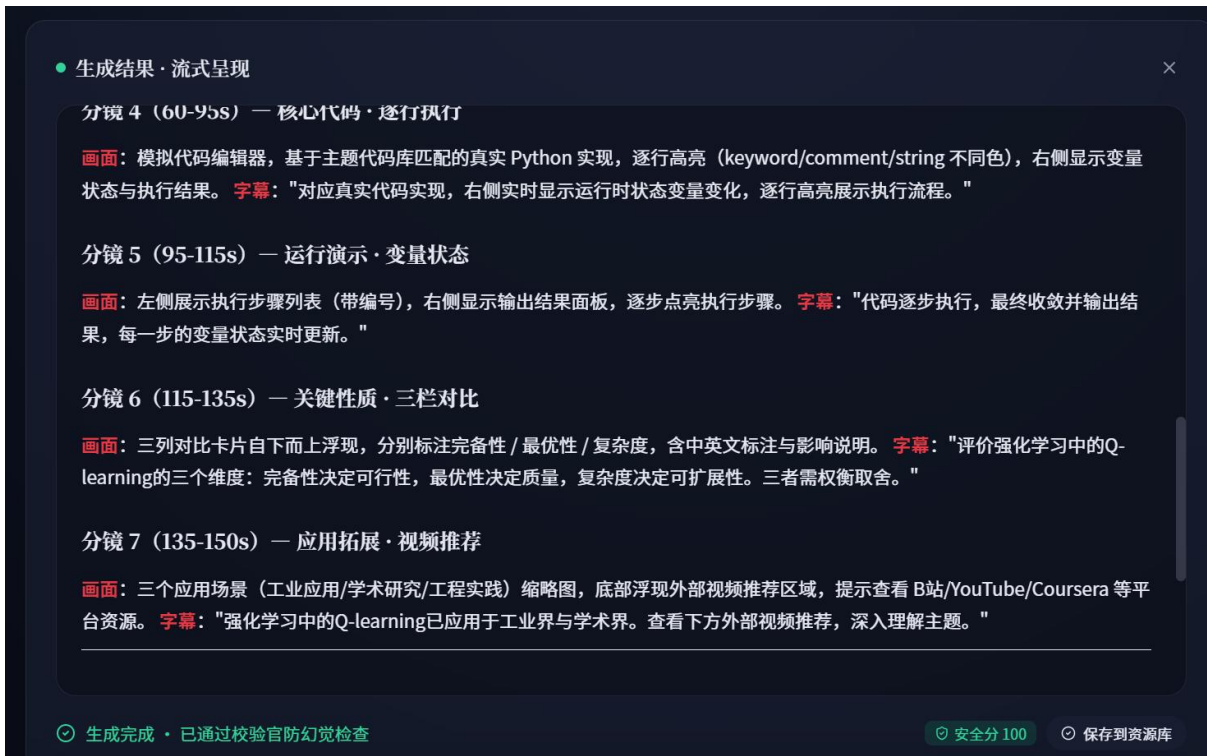


图 3.9 生成视频推导文本



图 3.10 生成视频成品内容截取



图 3.11 生成视频内容截取

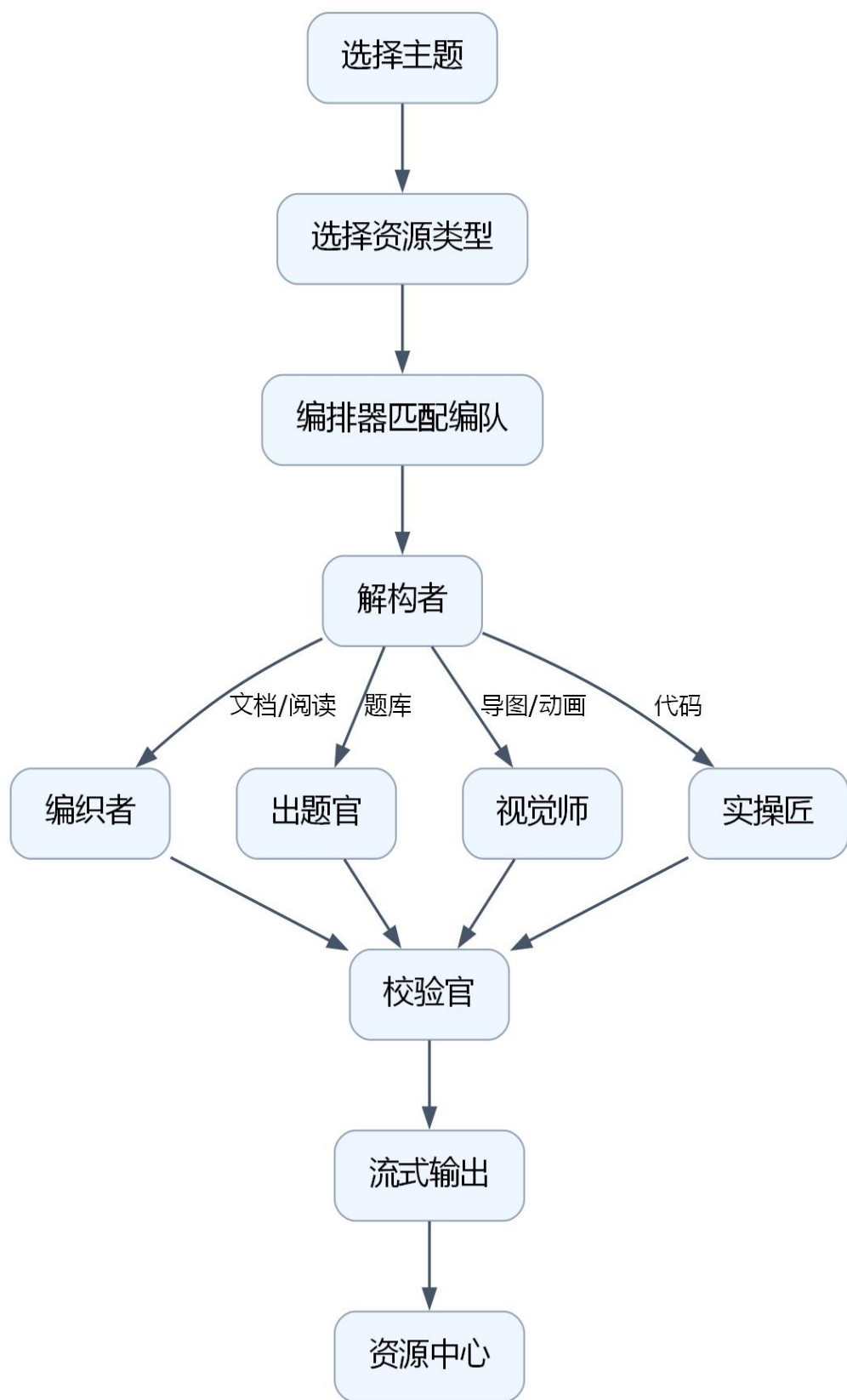


图 3.12 资源工坊模块处理流程图

核心代码说明：该片段摘取或概括自当前项目源码，体现模块实现的关键状态、流程或算法。

```
export async function* orchestrate(task, onLog) {
  const pipeline = pipelineLogTemplates[task.resourceType];
  onLog({ agentId: "captain", phase: "thinking", message: "正在调度智能体编队" });
  for (const stage of pipeline) {
    const agent = getAgentById(stage.agentId);
    onLog({ agentId: agent.id, phase: "generating", message:
stage.phase });
    await sleep(500);
  }
  const resource = buildCustomResource(task.resourceType, task.topic);
  const safety = validate(resource.content, task.knowledgeId);
  for await (const tok of streamText(resource.content)) yield tok;
  return { resource: { ...resource, safetyCheck: safety } };
}
```

功能运行时，页面层只负责采集用户操作和展示结果，核心状态变更集中由 **Store** 或服务函数处理。这样的划分让页面逻辑保持轻量，也便于后续接入真实后端接口、大模型服务或学习数据仓库。

3.5 学习路径模块

对应文件：LearningPath.tsx / course.ts / types.ts

学习路径模块以知识节点为基本单位，展示未学、进行中、已完成、推荐等状态。路径既依赖课程知识图谱，也会读取画像和学习记录，让系统知道哪些节点可以跳过，哪些节点需要补强，哪些节点适合作为下一步推荐。

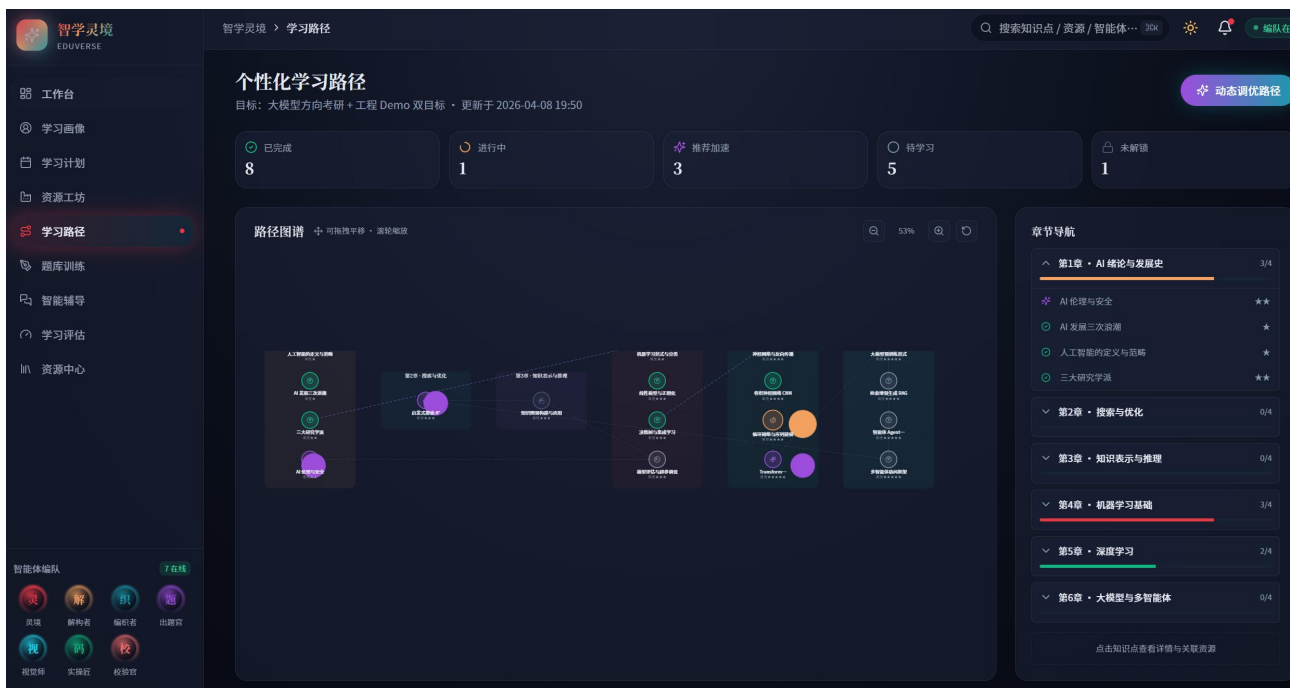


图 3.13 学习路径界面截图

核心代码说明：该片段摘取或概括自当前项目源码，体现模块实现的关键状态、流程或算法。

```
export interface PathNode {
  knowledgeId: string;
  status: "locked" | "todo" | "doing" | "done" | "recommended";
  order: number;
  reason?: string;
}

export interface LearningPath {
  id: string;
  learnerId: string;
  goal: string;
  nodes: PathNode[];
}
```

功能运行时，页面层只负责采集用户操作和展示结果，核心状态变更集中由 **Store** 或服务函数处理。这样的划分让页面逻辑保持轻量，也便于后续接入真实后端接口、大模型服务或学习数据仓库。

3.6 题库训练模块

对应文件：`Practice.tsx` / `practice.ts` / `useAppStore.ts`

题库训练模块覆盖单选、多选、填空、判断和代码补全题型。题目与知识点绑定，学生完成训练后，系统记录答题数量、正确数、准确率和日期，用于后续评估与学习计划调整。

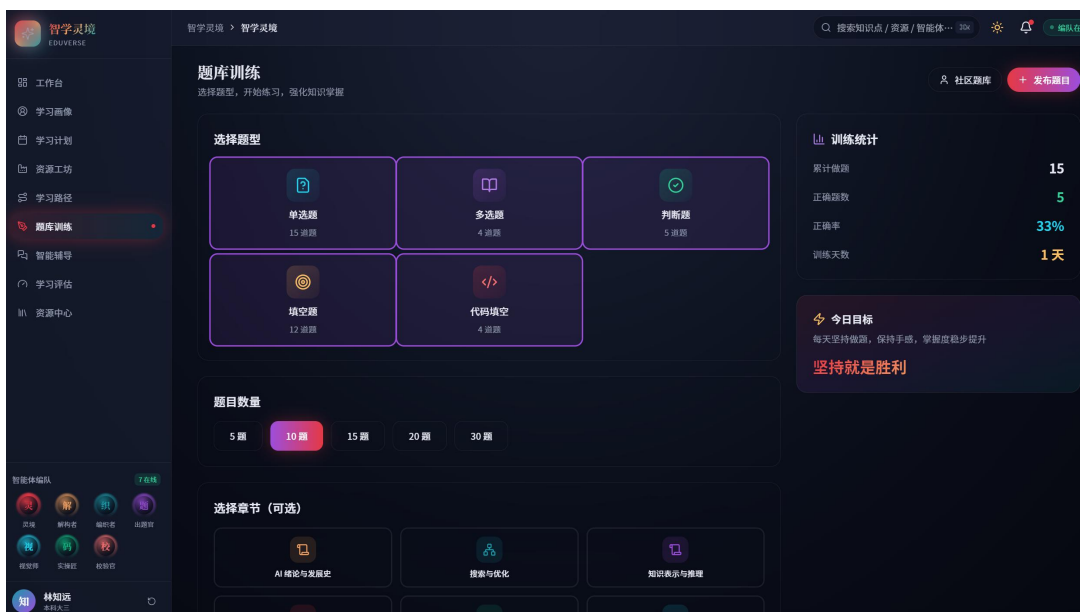


图 3.14 题库训练界面截图

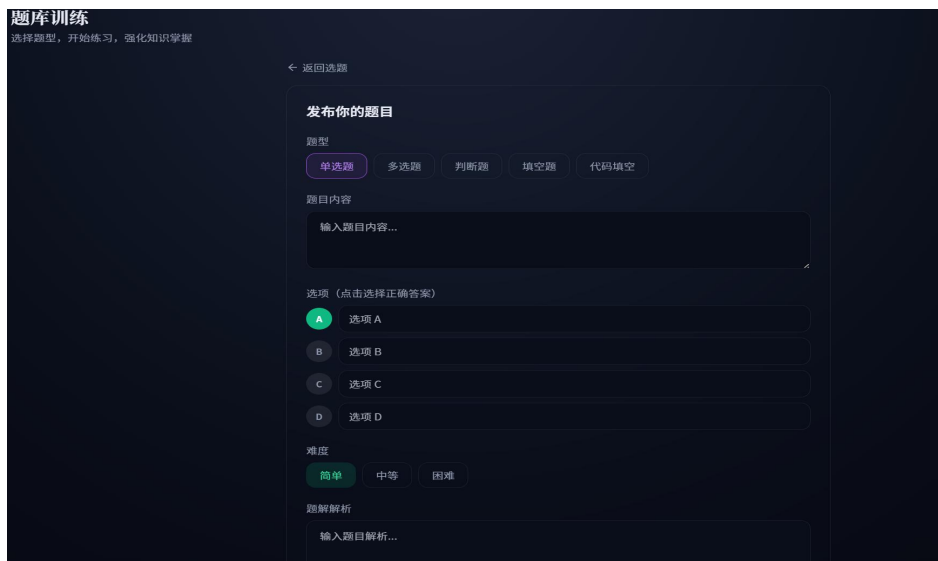


图 3.15 自行发布题目模块

核心代码说明：该片段摘取或概括自当前项目源码，体现模块实现的关键状态、流程或算法。

```
addPracticeRecord: (record) => {  
  const accuracy = record.totalQuestions > 0  
    ? Math.round((record.correctCount / record.totalQuestions) * 100)  
    : 0;  
  set((state) => ({  
    practiceRecords: [  
      { ...record, date: new Date().toISOString(), accuracy },  
      ...state.practiceRecords,  
    ],  
  })));  
}
```

功能运行时，页面层只负责采集用户操作和展示结果，核心状态变更集中由 **Store** 或服务函数处理。这样的划分让页面逻辑保持轻量，也便于后续接入真实后端接口、大模型服务或学习数据仓库。

3.7 智能辅导模块

对应文件：Tutor.tsx / tutor.ts / streaming.ts

智能辅导模块先从问题中识别关键词，再匹配课程知识点或补充概念，最后生成包含核心定义、关键性质、图示理解、代码实现、常见易错点和学习路径建议的结构化回答。它适合解释 **A***、**Transformer**、梯度爆炸、残差连接等具体知识点。

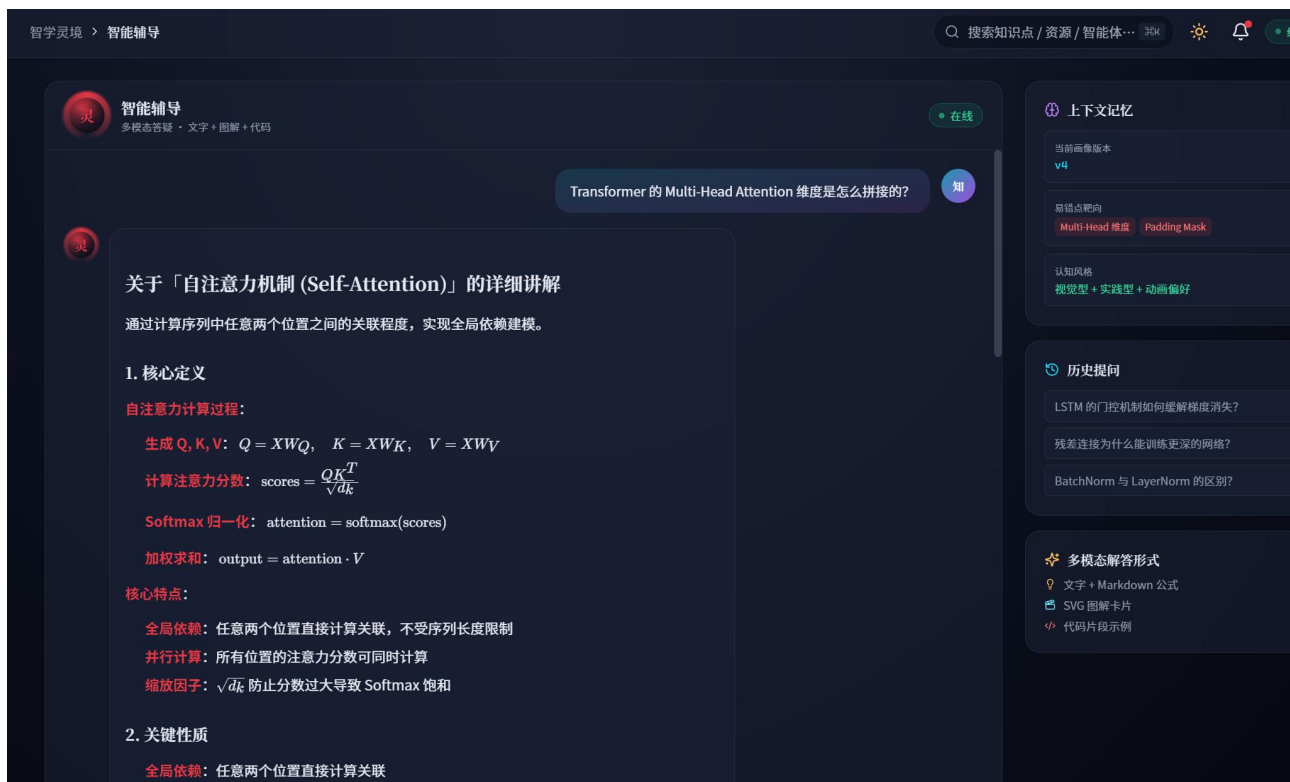


图 3.16 智能辅导界面截图

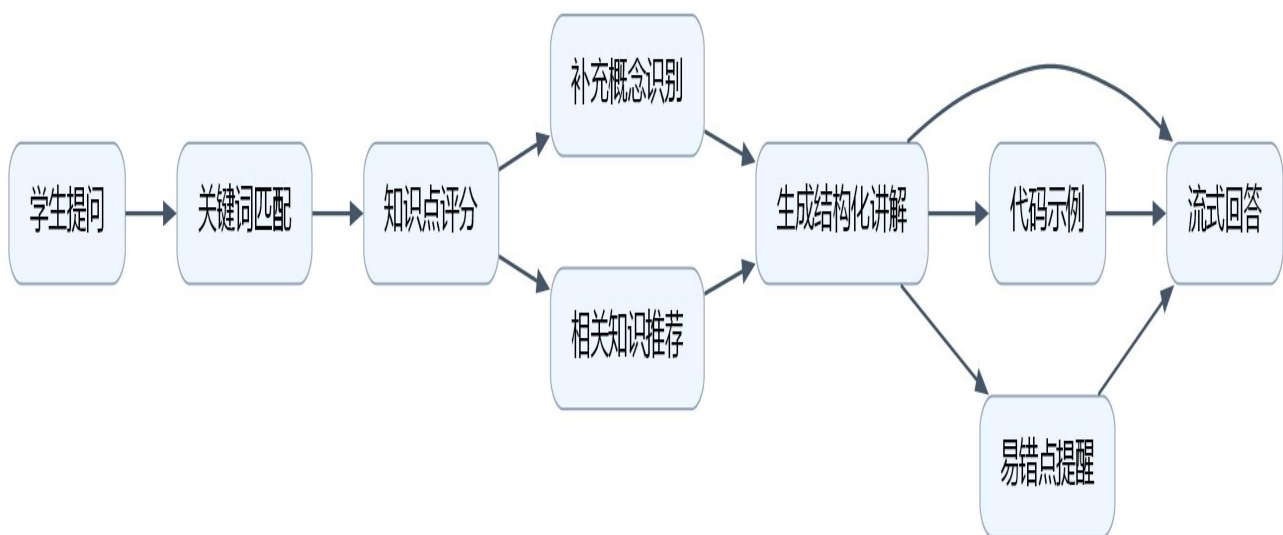


图 3.17 智能辅导模块处理流程图

核心代码说明：该片段摘取或概括自当前项目源码，体现模块实现的关键状态、流程或算法。

```

export function matchKnowledge(question: string) {
  let bestScore = 0;

```

```

let bestId = null;
for (const [id, keywords] of Object.entries(keywordMap)) {
  const score = similarityScore(question, keywords);
  const node = knowledgeNodes.find((k) => k.id === id);
  if (node && score > bestScore) {
    bestScore = score;
    bestId = id;
  }
}

return { primary: knowledgeNodes.find((k) => k.id === bestId) };
}

```

功能运行时，页面层只负责采集用户操作和展示结果，核心状态变更集中由 **Store** 或服务函数处理。这样的划分让页面逻辑保持轻量，也便于后续接入真实后端接口、大模型服务或学习数据仓库。

3.8 学习评估模块

对应文件：Assessment.tsx / RadarChart.tsx / useAppStore.ts

学习评估模块把学习记录、练习正确率、画像版本和资源使用行为汇总为评估结果。展示层使用雷达图、热力图和建议卡片，帮助学生理解自己在哪些维度进步、哪些知识点仍需要复习。

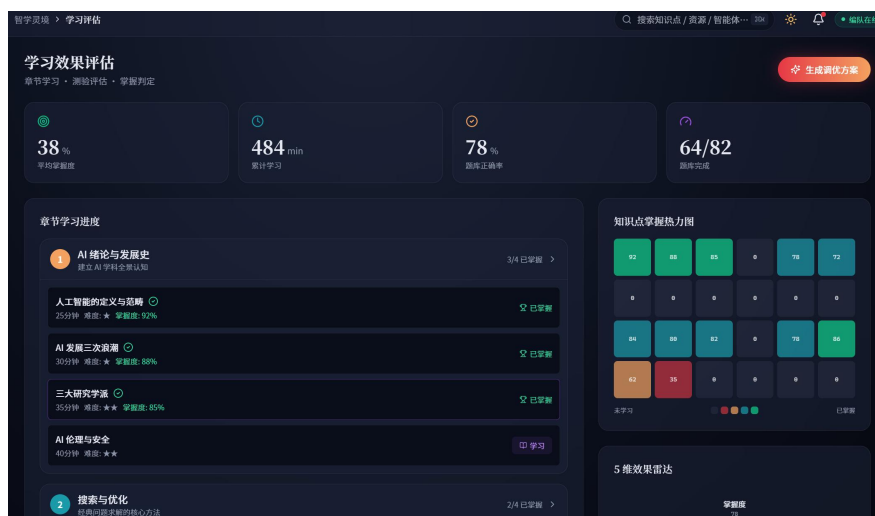


图 3.18 学习评估界面截图

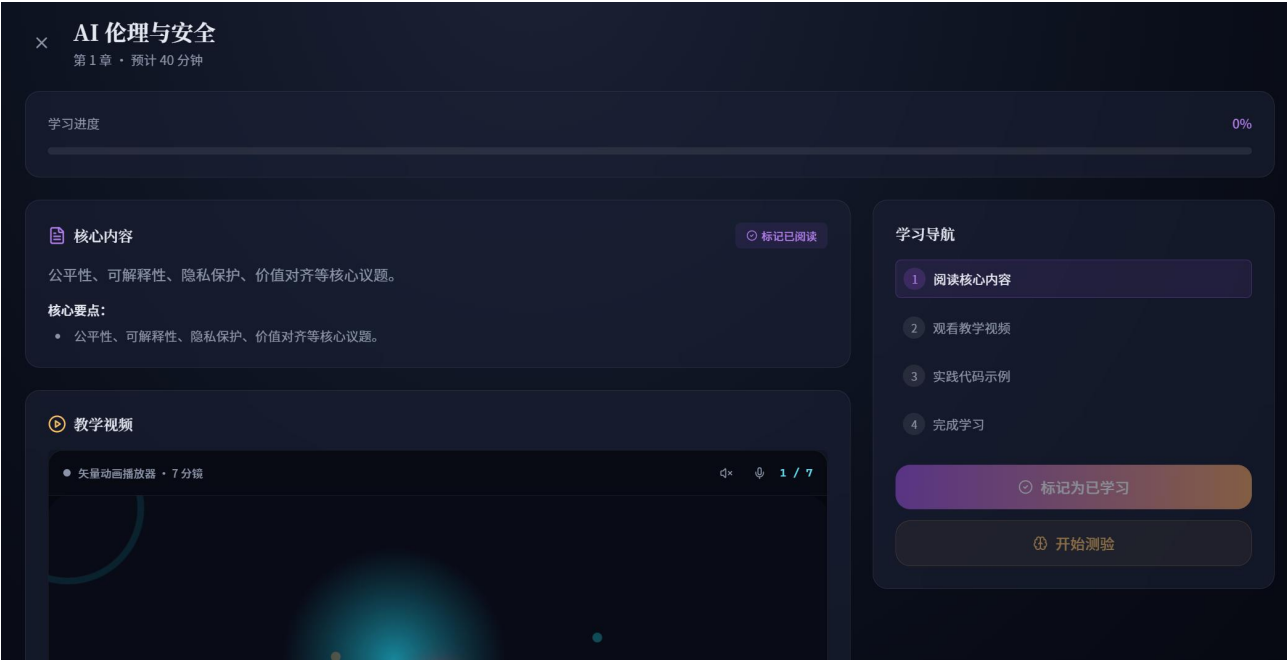


图 3.19 学习详情界面



图 3.20 章节测试详情

核心代码说明：该片段摘取或概括自当前项目源码，体现模块实现的关键状态、流程或算法。

```
getTotalPracticeStats: () => {  
  const records = get().practiceRecords;  
  const totalQuestions = records.reduce((sum, r) => sum + r.totalQuestions,  
0);
```

```
const correctCount = records.reduce((sum, r) => sum + r.correctCount, 0);

const accuracy = totalQuestions > 0 ? Math.round(correctCount /
totalQuestions * 100) : 0;

return { totalQuestions, correctCount, accuracy, days: new
Set(records.map(r => r.date)).size };

}
```

功能运行时，页面层只负责采集用户操作和展示结果，核心状态变更集中由 **Store** 或服务函数处理。这样的划分让页面逻辑保持轻量，也便于后续接入真实后端接口、大模型服务或学习数据仓库。

3.9 资源中心模块

对应文件：Library.tsx / ResourceCard.tsx / resources.ts

资源中心统一收纳资源工坊生成或预置的全部学习资源，支持按资源类型、知识点和收藏状态浏览。它让学生可以在学习路径之外重新查找讲解文档、导图、题库、拓展阅读、动画和代码案例。

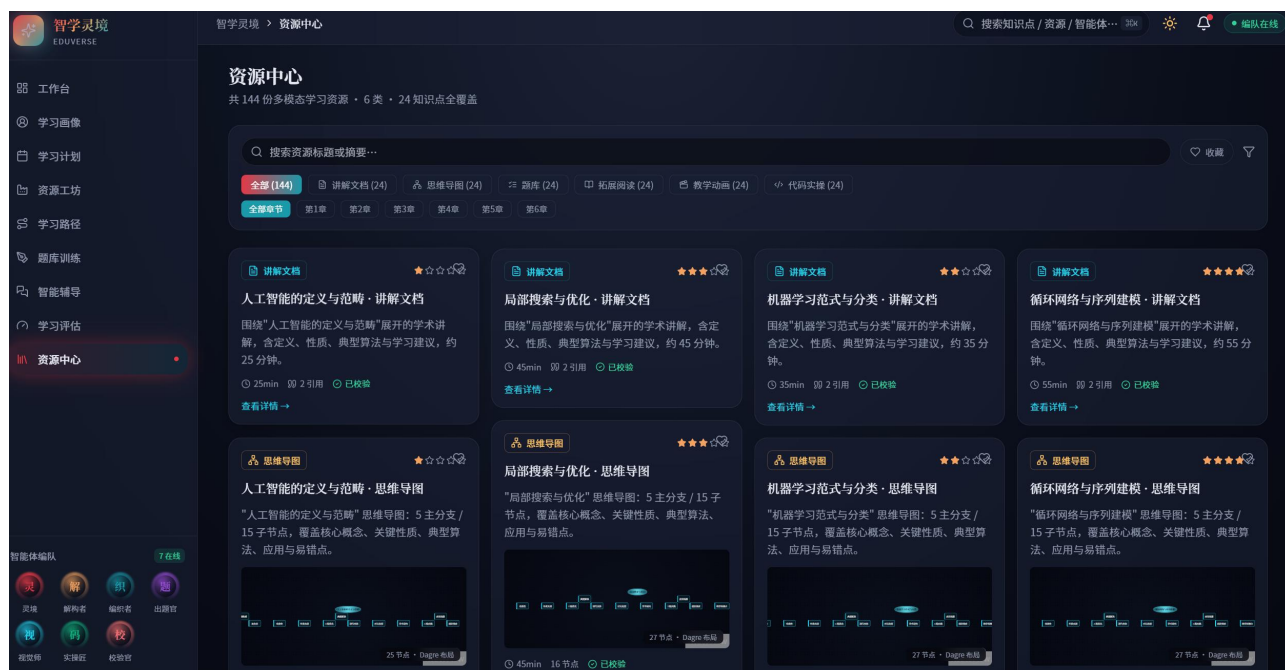


图 3.21 资源中心界面截图



图 3.22 讲解文档



图 3.23 思维导图



图 3.24 多样类型

核心代码说明：该片段摘取或概括自当前项目源码，体现模块实现的关键状态、流程或算法。

```
likedResource: (id) => set((state) => ({
  resources: state.resources.map((r) =>
    r.id === id ? { ...r, liked: !r.liked } : r
  ),
}));
```

功能运行时，页面层只负责采集用户操作和展示结果，核心状态变更集中由 **Store** 或服务函数处理。这样的划分让页面逻辑保持轻量，也便于后续接入真实后端接口、大模型服务或学习数据仓库。

四、模块详细设计

本章不再按单个页面重复描述，而是将工作台、学习画像、学习计划、资源工坊、学习路径、题库训练、智能辅导、学习评估和资源中心统一归纳为四个设计维度。这样既能覆盖全部页面模块，又能避免同类输入、状态、异常处理和测试方法反复出现。

表 4.1 设计层次

设计维度	覆盖模块	核心设计内容	源码依据	测试关注点
页面交互 维度	工作台、学习画像、学习计划、学习路径、题库训练、智能辅导、学习评估、资源中心	负责把学习状态、资源、路径、题目和对话转化为可操作界面。页面层只处理点击、输入、筛选、跳转、展开抽屉、提交答案等交互，不直接承担复杂业务计算。	src/pages 下各页面组件， src/components 下的 RadarChart、ResourceCard、MindmapViewer、StreamingText 等展示组件	侧边栏跳转是否正常；深浅主题切换后卡片、图表和代码块是否溢出；截图中的关键入口是否能在演示时稳定呈现
状态数据 维度	学习画像、学习计划、题库训练、学习评估、资源中心	负责沉淀学习者信息、画像版本、资源列表、学习记录、练习记录、计划任务和对话消息。所有模块共享同一个状态源，使画像、路径、题库和评估能够互相读取数据。	src/store/useAppStore.ts， src/types/index.ts， src/data/learner.ts， src/data/practice.ts， src/data/resources.ts	刷新页面后 LocalStorage 是否恢复；画像版本、练习记录和收藏状态是否丢失；新用户无历史数据时是否有默认兜底

智能生成 维度	资源工坊、 智能辅导、 学习路径、 学习计划	负责将学生的主题、 问题或学习目标转化 为多智能体任务。资 源工坊通过编排器生 成六类资源，智能辅 导通过知识点匹配生 成结构化讲解，学习 计划和学习路径根据 画像与记录给出后续 安排。	src/agents/orchestrator .ts, src/data/agents.ts, src/services/tutor.ts, src/services/streaming. ts, src/data/course.ts	资源生成日志 是否完整；流 式输出是否中 断；未命中知 识点时是否返 回引导性回 答；不同资源 类型是否调用 对应智能体编 队
质量保障 维度	所有生成类 模块与学习 结果类模块	负责处理异常、校验 内容可信度并支撑后 续扩展。校验官检查 敏感词、知识点白名 单、代码块和引用提 示；系统对空输入、 资源缺失、知识点未 命中和存储失败提供 回退。	src/services/safety.ts, src/services/streaming. ts, src/store/useAppStore. ts 中的 loadFromStorage、 saveToStorage、 validate、update 系列 函数	输入敏感词或 边界主题时是 否提示风险； 代码块格式异 常是否被识 别；资源缺失 是否回退；后 续接入后端和 真实大模型时 接口边界是否 清晰

页面交互维度强调“页面只管呈现和触发”。例如工作台展示学习概览和快捷入口，学习画像展示雷达图和画像历史，题库训练展示题目和解析，资源中心展示资源筛选与收藏。它们的共同点是把状态变成可读界面，而不是在页面组件里写复杂算法。

状态数据维度强调“所有学习行为都要留下证据”。画像变化需要记录触发原因，题库训练需要记录知识点、题型、正确数和准确率，资源中心需要记录收藏状态，学习评估需要能追溯到练习记录和学习记录。这样系统才能从一次性展示工具变成可持续更新的学习档案。

智能生成维度体现项目的多智能体特色。资源工坊按资源类型选择不同编队，智能辅导先做知识点匹配再组织回答，学习路径和学习计划读取画像与记录后给出推荐。演示时可以重点说明“学生输入并不直接生成页面结果，而是先进入编排器或服务函数，再由智能体链路输出内容”。

质量保障维度对应系统的可信边界。当前版本用轻量化校验机制保证演示稳定，后续可替换为教材 **RAG**、代码沙箱、内容安全 **API**、教师审核和后端日志审计。这样既保留 **Demo** 的一键运行能力，也为正式部署预留了工程扩展空间。

表 4.2 工程维度

维度	典型输入	典型输出	异常边界	后续扩展
页面交互	点击菜单、切换主题、选择资源类型、打开节点抽屉	卡片、图表、截图所示页面状态、弹层和抽屉	页面白屏、组件溢出、按钮无反馈	移动端适配、教师端、班级看板
状态数据	画像对话、练习结果、收藏操作、计划参数	画像版本、练习记录、资源状态、计划任务	LocalStorage 解析失败、空历史数据、版本冲突	后端数据库、用户登录、多端同步
智能生成	学习主题、问题文本、学习目标、知识点选择	讲解文档、思维导图、题库、代码案例、辅导回答	知识点未命中、资源模板缺失、流式输出中断	真实大模型、 RAG 检索、 SSE/WebSocket
质量保障	生成内容、代码块、知识点 ID、敏感词边界输入	安全分、风险标记、校验通过或人工复核提示	敏感词命中、白名单外知识点、代码格式异常	内容审核 API 、代码沙箱、教师审核流

五、 关键算法说明

5.1 画像增量更新算法

画像增量更新算法的核心逻辑是将学生学习过程中产生的文本、行为类自然语言学习信号，精准映射至学生画像各特征维度的数值增减，实现画像数据动态、持续迭代。系统预先为画像每一个评估维度（如知识基础、学习节奏、动手实操能力、兴趣偏好、理解速度等）维护独立的正向关键词词库与负向关键词词库，建立信号与维度变动的量化映射规则。以「知识基础」维度举例，对话、答题反馈、学习日志中出现“掌握、学会、吃透、进步、熟练”等正向词汇时，对应维度分值正向提升；若出现“卡住、不会、太难、听不懂、记不住”等负向表述，则下调该维度分数，同时同步调整学习节奏、理解难度适配度等关联维度。

本算法在演示阶段不追求与专业心理测量量表完全对齐，核心目标是搭建一套完全可解释、可溯源的动态画像更新机制。算法执行时会完整留存触发分值变动的原始文本、匹配关键词、增减数值，每一次画像维度的升降都能定位到明确的行为或文本证据。彻底解决传统自适应学习系统“黑盒更新”痛点，学生可随时查看画像变化原因，教师也能根据关键词日志精准定位学生学习卡点。

配套前端核心实现逻辑采用遍历维度关键词配置表的方式批量计算维度变动量：遍历全部画像维度配置项，初始化单维度变化值 **delta**；依次匹配该维度下所有正向关键词，若学习文本中包含对应词汇，则随机叠加 **5~10** 分增量；再遍历负向关键词库，匹配成功则扣除 **5~10** 分；最终将每个维度的总变化量存入维度差值对象，统一完成画像数值更新。随机分值区间用于模拟真实学习的梯度差异，避免每次相同反馈带来完全一致的画像变动，提升画像动态真实性。

```
for (const [dimName, config] of Object.entries(dimensionKeywords)) {
  let delta = 0;
  for (const kw of config.up) {
    if (text.includes(kw.toLowerCase())) delta += 5 +
Math.floor(Math.random() * 6);
  }
  for (const kw of config.down) {
```

```

        if (text.includes(kw.toLowerCase())) delta -= 5 +
Math.floor(Math.random() * 6);
    }
    dimensionDeltas[dimName] = delta;
}

```

5.2 知识点匹配算法

智能辅导模块底层采用关键词长度加权匹配算法完成学生提问与学科知识点的快速关联检索，是答疑、资源推送、错题溯源的基础支撑逻辑。算法会将学生输入的问题文本统一做小写标准化处理，分别和知识库内每条知识点绑定的专业关键词、知识点标题、教材摘要片段三组文本做字段比对，按照关键词字符长度累加匹配得分，最终选取总分最高的知识点作为当前提问对应的核心主知识点。确定主知识点后，系统读取学科知识图谱内章节从属关系、前置依赖、平行拓展知识点链路，批量输出关联学习内容推荐列表。

该轻量级文本匹配方案轻量化、无外部依赖，适配项目离线 **Demo** 演示场景：无需部署向量数据库、无需调用第三方大模型检索 **API**，仅通过本地字符串匹配即可完成知识点关联，降低部署门槛与运行成本。同时算法做了分层兼容设计，接口返回数据结构完全标准化，后续正式生产版本可无缝替换底层检索逻辑，上层业务功能无需改动。可替换升级方案包括稠密向量语义检索、**BM25** 全文检索、教材文本切片 **RAG** 检索、关键词 + 向量混合检索等高精度方案，兼顾演示便捷性与商业化落地扩展性。

核心相似度打分函数实现逻辑：统一标准化问题文本大小写，遍历知识点全部专业关键词，每匹配到一条关键词，就累加该关键词的字符长度作为匹配权重，最终输出总匹配得分，得分越高代表提问与该知识点关联越强。长专业术语（如卷积神经网络、傅里叶变换）权重更高，避免短通用词汇造成匹配偏差。

```

function similarityScore(question: string, keywords: string[]): number {
    const q = question.toLowerCase();
    let score = 0;
    for (const kw of keywords) {
        if (q.includes(kw.toLowerCase())) score += kw.length;
    }
}

```

```
return score;  
}
```

5.3 资源生成编队算法

资源生成编队算法是多智能体协同调度的核心调度逻辑，核心作用是根据用户请求的资源类型，自动分配、串联专属智能体处理链路，标准化多智能体协作流水线。系统预先定义不同学习资源对应的固定智能体执行编队流程：

理论讲解文档：解构者→编织者→校验官；解构者拆分知识图谱节点、梳理重难点，编织者整合图文、课件、推导步骤生成定制讲义，校验官完成知识点纠错、幻觉过滤；

分层题库习题：解构者→出题官→校验官；解构者拆解考点与易错分支，出题官基于学生画像难度分层生成选择、填空、编程习题，校验官核对答案正确性、规避超纲错题；

算法 / 电路可视化动画：解构者→视觉师→校验官；解构者梳理逻辑流程与参数关系，视觉师生成流程图、动态仿真图解、算法动效素材，校验官校验可视化逻辑与专业规范；

编程 / 硬件实操指导：解构者→实操匠→校验官；解构者拆分实操分步流程，实操匠输出代码示例、调试排错步骤、硬件接线指南，校验官验证代码可运行性、实操安全规范。

编队算法最大创新价值在于多智能体协作全流程可观测、可解释。编排器实时记录每条链路中各个智能体的运行状态，分为任务接收、思考解析、内容生成、安全校验、任务完成五类阶段并流式同步至前端界面。学生使用资源工坊时，可直观查看当前是哪一个智能体在处理内容；项目评审阶段，可通过完整调度日志清晰区分各智能体分工，直观验证系统真正实现多智能体分工协同，而非单一大模型直接输出内容。同时编队链路可动态拓展，新增资源类型时仅需配置对应智能体执行顺序，调度框架无需重构。

5.4 评估统计算法

学习评估统计算法融合题库练习时序记录、全周期学习行为日志、画像维度增量变化三类多源异构数据，完成数据清洗、指标聚合，为前端可视化评估页面提供量化数据源。三类数据各司其职，共同支撑评估模块可视化图表：

- 题库训练数据集：输出各知识点答题正确率、总作答题目量、错题频次、答题耗时等量化指标，作为热力图、错题分布统计图的底层数据；

- 自适应学习路径数据：记录各知识节点解锁状态、完成进度、重复学习次数、跳转回溯记录，用于展示学习完整流程进度；
- 学生画像时序数据：汇总每次增量更新后的维度数值，生成能力变化时序曲线，支撑多维度综合能力雷达图绘制。

算法摒弃传统学习平台仅输出总分、排名的静态评估模式，实现“评估 - 补学”闭环联动。在完成指标聚合、图表渲染后，算法自动筛选三类待补强数据：低正确率薄弱知识点、长期未完成的学习路径节点、持续下降的画像能力维度，并将上述目标自动关联知识库、资源池内配套讲义、习题、实操教程、可视化素材，生成个性化补强学习清单。让评估报告不再是单纯的数据展示页面，而是直接衔接后续针对性学习行为，形成“练习采集 - 综合评估 - 精准推送补强资源”的完整自适应学习闭环，持续推动学生补齐知识短板。

六、智能体协同机制

6.1 角色定义

表 6.1 角色定义

智能体	职责	触发场景
灵境队长	对话引导、画像构建、任务分发	首次进入、画像更新、资源生成入口
解构者	知识点拆解、前置依赖分析、导图骨架	文档、导图、路径、阅读、题库前置阶段
编织者	讲解文档和拓展阅读撰写	讲解文档、拓展阅读
出题官	题目生成、难度分级、易错点靶向	题库训练与资源工坊题库生成
视觉师	SVG 图解与教学动画分镜	思维导图、动画、可视化资源
实操匠	代码案例、运行环境、示例注释	代码实操资源
校验官	事实核查、安全过滤、引用补全	所有资源生成末段

编排器并不是把所有智能体同时调用，而是根据资源类型选择最短必要链路。讲解文档不需要出题官，代码实操不需要编织者，题库不需要视觉师。这种按任务选择编队的方式可以减少冗余步骤，也让协作日志更清晰。

6.2 编排器状态流转

```
const pipelineLogTemplates = {
  document: [
    { phase: "拆解知识结构", agentId: "deconstructor" },
    { phase: "撰写讲解文档", agentId: "weaver" },
    { phase: "事实核查与安全过滤", agentId: "validator" },
  ],
  quiz: [
```

```

    { phase: "锁定考点", agentId: "deconstructor" },
    { phase: "生成多梯度题目", agentId: "quizmaster" },
    { phase: "答案与解析校验", agentId: "validator" },
  ],
};

```

6.3 流式输出机制

流式输出通过 **async generator** 实现。内容先被 ``tokenize`` 切成文本片段，再由 ``streamText`` 按速度逐段 `yield` 到页面。标点处增加短暂停顿，使输出更接近真实 AI 对话体验，同时不会破坏 **Markdown** 代码块和公式结构。

```

export async function* streamText(text, opts = {}) {
  const speed = opts.speed ?? 18;
  let count = 0;
  for (const tok of tokenize(text)) {
    yield tok;
    count++;
    opts.onToken?.(count);
    const pause = /[。！？，、；：\n]/.test(tok) ? speed * 4 : speed;
    await sleep(pause);
  }
}

```

七、安全校验与防幻觉

智学灵境当前版本采用轻量化校验策略，目标是在演示阶段明确体现“生成后必须验证”的安全边界。校验官检查敏感词、知识点白名单、代码块粗语法和引用补全，得到安全分后才允许资源进入最终展示与入库流程。

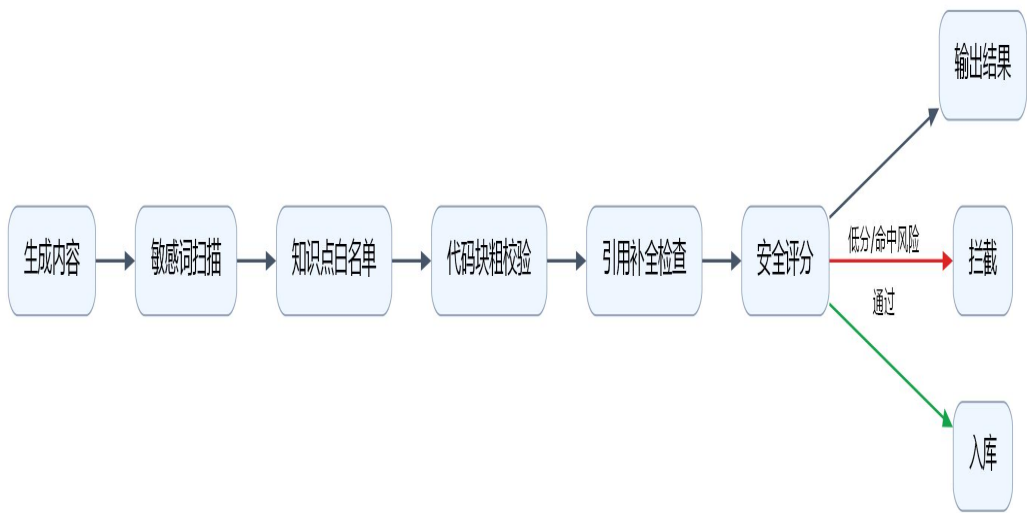


图 7.1 安全校验与防幻觉流程图

```
export function validate(content: string, knowledgeId?: string):
SafetyResult {
  const flags: string[] = [];
  for (const w of SENSITIVE_WORDS) {
    if (content.includes(w)) flags.push(`敏感词命中: ${w}`);
  }
  if (knowledgeId && !KNOWN_KNOWLEDGE_IDS.has(knowledgeId)) {
    flags.push("知识点不在白名单，需人工复核");
  }
  const codeBlocks = content.match(/```[\s\S]*?```/g) || [];
  for (const block of codeBlocks) {
    const inner = block.replace(/```\w*\n?/, "").replace(/```$/, "");
    if (inner.includes("def ") && !inner.includes(":")) {
      flags.push("代码块缺少冒号，疑似语法错误");
    }
  }
}
```

```

    }
  }

  const score = Math.max(0, 100 - flags.length * 18);
  return { passed: flags.length === 0, flags, score };
}

```

该校验方式适合竞赛与课程项目的轻量演示。生产版本可以在相同接口下替换为 **RAG** 检索、教材引用核对、模型输出置信度评估、代码沙箱执行和内容安全 **API**，从而增强可靠性。

八、系统测试

8.1 测试环境

环境	配置	作用
本地开发端	Windows 11、Node.js、npm、Chrome 或 Edge	运行 Vite 开发服务器，验证页面与交互
前端技术栈	React 18、TypeScript、Vite、Tailwind CSS	构建页面、状态、样式与动效
测试命令	npm run check / npm run build	检查 TypeScript 类型与生产构建
浏览器验证	页面截图与模块跳转	检查工作台、画像、工坊、路径、辅导、评估等页面

8.2 功能测试结果

编号	场景	输入或操作	预期结果	结论
TC-01	首次进入	进入系统并完成引导	生成初始画像并进入工作台	通过

TC-02	画像更新	输入“最近 Transformer 学得不错但 DP 仍会错”	画像维度发生增量变化并记录证据	通过
TC-03	资源生成	选择“代码实操 + A* 算法”	智能体日志滚动，内容流式输出并通过校验	通过
TC-04	智能辅导	询问“梯度爆炸和梯度消失区别”	返回定义、对比、解决方法和学习建议	通过
TC-05	题库训练	完成一组单选和代码补全题	生成准确率并进入练习记录	通过
TC-06	评估展示	查看学习评估页	雷达图、热力图和建议正常展示	通过

8.3 性能与稳定性

当前版本的核心逻辑在浏览器本地运行，主要性能压力来自动效、卡片渲染、Markdown 渲染和资源列表筛选。由于大模型调用采用剧本式流式回放，演示环境不会受到外部 API 网络波动影响，能保证课堂答辩和竞赛评审中的稳定性。

当切换到真实大模型服务后，建议将智能体编排、内容校验和资源入库迁移到后端服务，并使用 SSE 或 WebSocket 保留当前前端的流式体验。前端 Store 可继续作为本地缓存与交互层状态管理，避免页面层被接口细节污染。

九、 AI 智能引入说明、

9.1 本项目使用的开源依赖均遵循其原始协议，主要依赖包括：

依赖	协议	用途
React 18	MIT	UI 框架
Vite 6	MIT	构建工具
Tailwind CSS 3	MIT	样式方案
framer-motion 11	MIT	动效
zustand 5	MIT	状态管理
react-markdown 9	MIT	Markdown 渲染
lucide-react	ISC	图标库

9.2 AI Coding 工具

主要包括：

- 代码生成：组件骨架、类型定义、Mock 数据生成
- 文档撰写：PRD、技术架构、开发说明书、测试说明书
- 代码审查：TypeScript 类型检查、潜在问题排查

按赛题要求，AI 辅助工具选用科大讯飞星火系列。

十、应用前景与迭代规划

10.1 应用场景

高校课程教学是智学灵境最直接的应用场景。教师可以将课程知识点、实验要求和题库标准导入系统，学生通过画像与路径获得个性化学习资源，课堂则保留统一教学节奏与重点讲授。

竞赛与项目实训场景也适合落地。学生可以围绕一个主题快速生成讲解文档、代码案例、思维导图和训练题，并通过智能辅导补齐薄弱点。这使平台不仅是学习工具，也能成为项目备赛时的知识组织工具。

10.2 短期迭代

- 接入真实大模型服务，在保留剧本式演示模式的同时支持在线生成。
- 扩展课程知识库，从人工智能导论扩展到机器学习、深度学习、数据结构和计算机网络。
- 完善题库训练统计，增加错题本、题目收藏、复习提醒和按知识点组卷。

10.3 中期迭代

- 引入后端数据库，持久化画像版本、学习记录、资源生成结果和教师端课程配置。
- 增加教师端，支持课程知识点管理、班级学习看板、资源审核和题库发布。
- 强化防幻觉机制，接入教材 RAG、引用校验、代码沙箱和内容安全服务。

10.4 长期规划

长期来看，智学灵境可以从单课程演示系统发展为高校智慧教育平台中的个性化学习引擎。它可以服务课程预习、课堂辅助、课后复习、考试训练、竞赛备赛和项目实训，并通过持续积累的学习数据反哺教学设计。

当系统具备多课程、多角色和多数据源能力后，学生画像将不再局限于某门课程，而是能形成跨学期、跨课程的学习成长档案。届时系统的核心价值会从“生成资源”提升为“理解学生并持续调整教育供给”。